

PyraNet: A Multi-Layered Hierarchical Dataset for Verilog

1st Bardia Nadimi
Dept. of Computer Science & Eng.
University of South Florida
Tampa, Florida, United States
bnadimi@usf.edu

2nd Ghali Omar Boutaib
Dept. of Computer Science & Eng.
University of South Florida
Tampa, Florida, United States
ghaliomar@usf.edu

3rd Hao Zheng
Dept. of Computer Science & Eng.
University of South Florida
Tampa, Florida, United States
haozheng@usf.edu

Abstract—Recently, there has been a growing interest in leveraging Large Language Models for Verilog code generation. However, the current quality of the generated Verilog code remains suboptimal. This is largely due to the absence of well-defined, well-organized datasets with high-quality samples, as well as a lack of innovative fine-tuning methods and models specifically trained on Verilog. In this paper, we introduce a novel open-source dataset and a corresponding fine-tuning technique, which utilizes a multi-layered structure that we refer to as PyraNet. Our experiments demonstrate that employing the proposed dataset and fine-tuning approach leads to a more accurate fine-tuned model, producing syntactically and functionally correct Verilog code. The evaluation results show improvements by up-to 32.6% in comparison to the CodeLlama-7B baseline model and up-to 16.7% in comparison to the state-of-the-art models using VerilogEval evaluation platform.

Index Terms—Large Language Models, fine-tuning, Verilog, dataset, transformers

I. INTRODUCTION AND MOTIVATION

The introduction of attention-based models [1] represents a landmark development in the field of language processing. This breakthrough spurred the widespread adoption of transformer architectures, driving significant advancements in the area. Many models, such as Generative Pre-trained Transformers (GPT) [2], Bidirectional Encoder Representations from Transformers (BERT) [3], and Language Model for Dialogue Applications (LaMDA) [4], have leveraged transformer technology to achieve notable success.

The motivation behind utilizing large language models (LLMs) for hardware code generation is centered on developing a tool that streamlines the hardware modeling process for designers [5], [6]. It also enhances the security aspects of hardware design [7]. Integrating LLMs into this field aims to reduce the complexities traditionally involved in hardware design. By harnessing the advanced features of these models, hardware model development is expected to become more intuitive and efficient.

Additionally, the automation of hardware code generation through LLMs is vital for reducing human error. Given the intricate and technical nature of hardware design, manual coding is often prone to mistakes [8]. By automating the process, LLMs not only speed up development but also significantly lower the chances of introducing errors that could arise from human oversight. This results in more reliable and robust hardware designs, as the automated system ensures consistent, high-quality code generation with reduced risk of

faults. Ultimately, employing LLMs in this domain marks a move toward more efficient, error-resilient, and user-friendly approaches to hardware design.

While there has been extensive research focused on software program synthesis, hardware code synthesis remains relatively underexplored. Recently, researchers have begun to investigate the use of LLM architectures for generating Hardware Description Language (HDL) code [9]–[19].

Despite early successes in using LLMs for HDL code generation in hardware design, several challenges remain that must be addressed to push the field forward. One key issue is the limited availability of labeled data necessary for effective fine-tuning of LLMs, as seen in the more mature field of software code generation. Existing approaches to Verilog code generation often suffer from syntax and functionality errors [9], [10], highlighting the need for further improvements.

In this paper, we introduce PyraNet, a multi-layered open-source dataset structure ¹ that utilizes a wide range of open-source Verilog samples, irrespective of their code quality. By organizing the dataset into different quality tiers, we are able to maximize the value from both the high-quality samples and those of lower quality. The overall structure of the dataset is depicted in Fig.1-a.

The proposed fine-tuning technique is designed to fully leverage all data entries across the different tiers of the dataset. This is done by assigning higher loss weights to the high-quality data, and progressively reducing these weights for lower-quality data as we move down the pyramid [20]. Additionally, we implement a curriculum learning approach [21], starting the fine-tuning process with simpler data and gradually introducing more complex data as training progresses. By combining both the loss weighting and curriculum learning strategies, we can achieve a more effective fine-tuned model. The overall proposed fine-tuning architecture is depicted in Fig. 1. The **key contributions** of this paper is summerized as follows:

- The introduction of the PyraNet open-source dataset that includes various quality tiers, categorized based on the quality of the Verilog code, as well as the available labels for each entry. The labels include information such as the complexity level of the code, code rankings, design descriptions, and compile details.

¹<https://huggingface.co/datasets/bnadimi/PyraNet-Verilog>

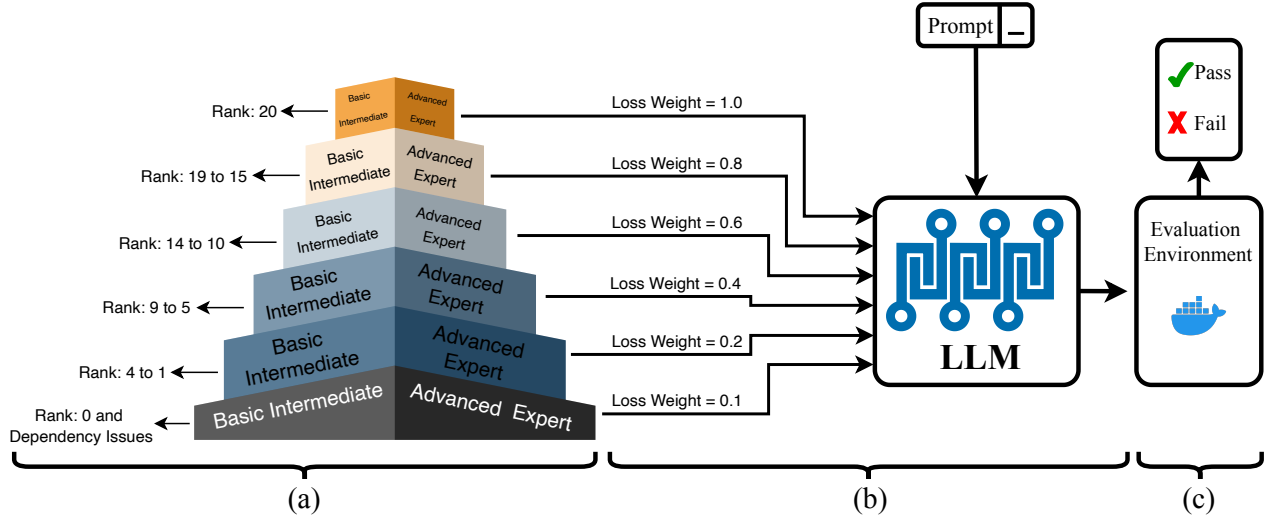


Fig. 1. Proposed Overall Architecture. (a) PyraNet dataset architecture. (b) PyraNet fine-tuning architecture. (c) Evaluation process.

- The introduction of the corresponding fine-tuning approach which is designed to maximize the benefits of the proposed PyraNet dataset by incorporating both the loss weighting technique and the curriculum learning strategy.
- Experiments show that the new PyraNet approach improves the Verilog code generation by up-to 32.6% in comparison to the CodeLlama-7B baseline model and up-to 16.7% in comparison to the state-of-the-art approaches including RTLCode [18] and OriGen [22].

The remainder of this paper is organized as follows: Section II reviews related research in the field. Section III introduces the methodology, along with the proposed dataset and fine-tuning approach. Section IV focuses on comparisons and evaluations. Lastly, conclusions and future directions are discussed in Section V.

II. RELATED WORKS

The study in [9] explores the capability of LLMs to generate Verilog code for hardware design, demonstrating significant success in achieving syntactically accurate outputs through fine-tuning on Verilog-specific datasets. By emphasizing the role of LLMs in reducing human errors and enhancing automation in hardware design, the research underscores the need for further advancements in functional correctness and sets the stage for broader AI integration in hardware development.

The study in [10] presents VerilogEval, a benchmarking framework for evaluating the performance of LLMs in generating Verilog code for hardware design and verification. Utilizing a dataset of 156 problems from HDLBits, it investigates the automation of Verilog code generation across varying complexities, from simple circuits to advanced finite-state machines. The findings demonstrate that fine-tuning LLMs improves the quality of generated code, highlighting the transformative role of AI in optimizing hardware design workflows. Additionally, the research emphasizes the potential of supervised fine-tuning to enhance LLM capabilities, representing a significant step forward for both academic research and practical applications in the domain.

The study in [12] addresses the absence of standardized benchmarks for evaluating LLMs, such as ChatGPT, in hardware design, with a focus on Register Transfer Level (RTL) generation from natural language. It introduces RTLML, an open-source benchmark designed to evaluate the effectiveness of LLMs in RTL design, establishing new standards for syntax, functionality, and design quality. The study also proposes a novel prompt engineering technique, "self-planning," which enhances GPT-3.5's performance, representing a major advancement in leveraging LLMs for complex and scalable hardware design tasks.

MG-Verilog [23] introduces a multi-grained dataset tailored for LLM-assisted Verilog code generation, addressing limitations in size, complexity, and description granularity found in existing hardware datasets. MG-Verilog includes over 11,000 Verilog code samples with corresponding descriptions at varying levels of detail, such as high-level summaries, block summaries, and line-by-line comments. The dataset facilitates balanced fine-tuning, enabling LLMs to improve both code implementation accuracy and generalization across diverse hardware design tasks. Extensive experiments demonstrate the dataset's effectiveness, showcasing superior performance compared to models trained on other datasets, with enhanced metrics in code generation accuracy and functionality.

RTLCode [18] introduces a fully open-source framework for RTL code generation, addressing the challenges of privacy concerns and the lack of high-performance open-source LLMs for Verilog generation. The framework includes a dataset of over 27,000 instruction-code pairs, generated through an automated pipeline that ensures diversity and complexity across a wide range of hardware design tasks. RTLCode employs a lightweight LLM architecture with only 7 billion parameters, fine-tuned using a novel training scheme that incorporates code quality feedback to enhance generation accuracy.

MEV-LLM [24] presents a multi-expert large language model architecture tailored for Verilog code generation, addressing challenges of syntax and functionality errors in exist-

ing approaches. MEV-LLM employs multiple expert models, each fine-tuned to handle specific design complexity levels, ranging from basic to expert-level Verilog designs. The study develops a categorized dataset with fine-grained and coarse-grained labels to enhance the fine-tuning process, enabling better alignment between input descriptions and generated code. Evaluation against state-of-the-art models demonstrates significant improvements, with MEV-LLM achieving up to a 23.9% increase in the pass@k metric.

The paper [22] introduces OriGen, an open-source framework for Register Transfer Level (RTL) code generation, designed to address the limitations of existing approaches in data quality and self-reflection. OriGen employs a novel code-to-code augmentation methodology to generate high-quality RTL datasets and incorporates a self-reflection mechanism for error correction using compiler feedback. The framework trains two specialized LoRA (Low-Rank Adaptation) models, one for generating initial RTL code and another for rectifying syntactic errors, enhancing its ability to handle complex Verilog tasks. Experimental results show that OriGen outperforms all existing open-source LLMs, achieving a 12.8% improvement over the best-performing model on the VerilogEval-Human benchmark and demonstrating superior self-reflection capabilities, surpassing GPT-4 Turbo by 19.9% in syntactic correctness.

CodeV [25] presents a robust framework for Verilog code generation by leveraging multi-level summarization and fine-tuning large language models (LLMs). The approach constructs a high-quality dataset of 165,000 Verilog modules collected from real-world sources, enriched by GPT-3.5-generated functional and high-level summaries. The experimental results demonstrate state-of-the-art performance on benchmarks like VerilogEval and RTLLM, with CodeV surpassing GPT-4 by 22.1% on VerilogEval and achieving notable gains in syntax and functional correctness. This work highlights the potential of combining domain-specific datasets and advanced fine-tuning techniques to push the boundaries of hardware description language generation.

BetterV [26] introduces a framework for controlled Verilog generation that enhances Verilog implementation through a novel combination of instruct-tuning and generative discriminators. BetterV utilizes a domain-specific dataset, created through rigorous filtering and augmentation techniques, and aligns Verilog with C programs to enhance large language models' (LLMs) understanding of hardware description languages. By employing task-specific generative discriminators, BetterV optimizes downstream Electronic Design Automation (EDA) tasks such as synthesis node reduction and verification runtime. The framework achieves state-of-the-art performance on the VerilogEval benchmark, outperforming GPT-4 in functional correctness.

AutoVCoder [27] addresses challenges in register-transfer level (RTL) code generation through three key innovations: a high-quality hardware dataset generation strategy, a two-round fine-tuning process, and a domain-specific retrieval-augmented generation (RAG) mechanism. Experimental eval-

uations demonstrate AutoVCoder's superiority over existing state-of-the-art models, achieving notable improvements across multiple benchmarks, including up to 3.4% gains in functional correctness on the RTLLM benchmark. By integrating domain-specific retrieval and fine-tuning techniques, AutoVCoder bridges the gap between general-purpose LLM capabilities and the specialized requirements of RTL code generation, paving the way for efficient and high-quality hardware design automation.

Each of the previously discussed studies faces challenges in producing Verilog codes that are both syntactically and functionally accurate.

III. METHODOLOGY AND DATASET

This section starts by explaining the structure and organization of the PyraNet dataset, followed by a detailed description of the proposed fine-tuning process.

A. PyraNet Dataset

1) *Dataset gathering*: The majority of our dataset was gathered from publicly accessible GitHub repositories. Additionally, some Verilog code examples were generated using commercial large language models like GPT-4o-mini.

2) *Post-download dataset filters*: Following the collection of all Verilog code samples, we implemented multiple filtering procedures to guarantee the dataset's quality and relevance. The following are the filtration steps:

- **Empty/Broken files**: The initial step involved the removal of empty and corrupted or broken files. A Python script was employed to detect files with encoding issues by attempting to read each file. Any files that could not be processed due to encoding errors were discarded. Likewise, files that were successfully read but were empty were also excluded from the dataset.
- **Module declaration**: Files without a valid module declaration were similarly filtered out. Using a Python script, we identified and excluded any files that lacked module declarations, following the same procedure as the check for empty or corrupted files.
- **Code Deduplication**: We employed the Jaccard similarity algorithm to perform deduplication. This method computes the similarity between sets of tokens derived from the code samples by measuring the intersection over the union of the sets. Code pairs with a Jaccard similarity score above a predefined threshold were identified as duplicates and subsequently removed, ensuring the dataset's integrity and reducing redundancy.
- **Syntax Check**: The most crucial filtering step involved checking for syntax errors, as a dataset containing such errors would cause significant problems when training a model. To mitigate this, each file was processed using Icarus Verilog [28], and files that failed due to syntax errors were discarded. Files containing other types of errors, such as missing imports or undefined references, were also labeled under "dependency issues" since these do not compromise the syntactic correctness of the code. Given that running Icarus Verilog is more computationally demanding than the earlier filtering steps, this step

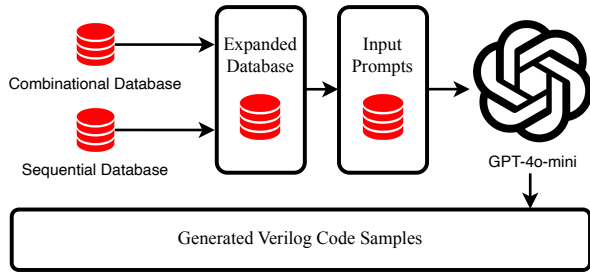


Fig. 2. Process for generating Verilog code samples using commercial LLMs.

was performed last, ensuring it was applied to a reduced subset of files.

3) *Verilog code generation using commercial LLMs:* In addition to the collected code samples, we also generated a set of code samples using commercial LLMs, such as GPT-4o-mini. To guide this process, we first compiled a database of keywords and categorized them into combinational and sequential circuits, focusing on general hardware and Verilog design terms such as adders, multipliers, counters, FSMs, etc. For each keyword, we expanded the database by detailing specific variations, for instance, ripple carry adders or carry-save adders—this step was referred to as “expanded-keywords.” Finally, for each expanded keyword, we crafted input prompts with detailed design descriptions and queried GPT-4o-mini 10 times for each prompt using different temperature values. Fig. 2 depicts the overall process for the generating the Verilog code samples.

4) *Ranking and Complexity labels:* Each entry in the PyraNet dataset was evaluated and ranked using the GPT-4o-mini model. Rankings were assigned on a scale from 0 to 20, where 0 denotes the lowest score and 20 the highest. To facilitate the ranking process, we generated specific input prompts for each Verilog code sample in the dataset. The GPT-4o-mini model was instructed to score each code based on guidelines provided in a pre-prompt text, which detailed the scoring methodology. The scoring criteria focused on the overall Verilog coding style and the efficiency of the code. Fig.3 presents an example of a prompt along with its corresponding response.

To assign a complexity level to each data sample in the PyraNet dataset, we closely followed the methodology presented in the MEV-LLM work [24]. This approach allowed us to generate and categorize the dataset into four distinct tiers: Basic, Intermediate, Advanced, and Expert.

5) *Dataset Organization:* We organized the collected and generated Verilog code samples into six distinct layers:

Layer 1 comprises a total of 235 data samples ranked 20 out of 20, representing the highest tier of Verilog codes. These samples are distinguished by their efficiency, coding style, and syntactic correctness.

Layer 2 includes 150,279 data samples with rankings ranging from 19 to 15.

Layer 3 contains a total of 105,973 data samples with rankings between 14 and 10.

Prompt:

Act as a teacher and rank the quality of this Verilog code in scale of 0 to 20, with 0 being syntactically incorrect and 20 being a good Verilog code in terms of efficiency and coding style:

```

module halfAdder (
    input A,
    input B,
    output Sum,
    output Cout
);

    assign Sum = A ^ B;
    assign Cout = A & B;
endmodule
  
```

Just give me the score only.

Response:

Score: 20 out of 20.

Fig. 3. Example of a prompt and the response used for ranking.

Layer 4 includes 5,015 data samples with rankings ranging from 9 to 5.

Layer 5 contains 275 data samples with rankings between 4 and 1.

It is important to note that for Layers 1 through 5, we ensured representation across all complexity levels. Additionally, all samples in these layers successfully compiled without errors.

Layer 6 contains a total of 430,461 data samples with dependency issues that could not be compiled without errors or ranking of 0.

Initially, we collected approximately 2.4 million code samples from open-source GitHub repositories and generated an additional 150,000 samples using GPT-4o-mini. Following the pre-processing phase, we obtained 692,238 code samples that were deduplicated, ranked, described, and compiled without syntax errors or having only dependency issues.

B. PyraNet fine-tuning

To effectively leverage the proposed PyraNet dataset, we have introduced a dedicated fine-tuning architecture. This architecture incorporates two key techniques: loss weighting and curriculum learning. In this section, we will first elaborate on the loss weighting approach, followed by a detailed explanation of the curriculum learning strategy.

1) *Loss Weighting:* As previously described, the PyraNet dataset is structured into six layers of data samples, each corresponding to different tiers. In our loss weighting technique, we fine-tune the model by assigning distinct loss weights to each tier within the dataset. Starting with the top layer—which contains highly ranked and high quality code samples—we set the loss weight to 1.0. This maximal weight ensures

that entries in this layer have the most significant impact on the fine-tuning process. As we descend through the layers of the PyraNet dataset, the loss weights are progressively decreased. This reduction lessens the influence of data entries from lower tiers on the final fine-tuned model. Consequently, the model prioritizes learning from high-quality data samples while minimizing the effects of those with lower quality. For the layer of 2 to 6 the loss weights are 0.8, 0.6, 0.4, 0.2, and 0.1 respectively, as depicted in Fig.1-b.

2) *Curriculum Learning*: As demonstrated in [21], employing a meaningful training sequence—beginning with simpler samples and progressing to more complex ones—can enhance model performance compared to standard training methods that use random sampling. In our proposed PyraNet dataset, we have assigned complexity levels to all code samples within each tier. Considering the benefits of curriculum learning, we propose initiating the fine-tuning process with the `basic` complexity level for each tier, followed by `intermediate`, `advanced`, and finally `expert` levels.

The fine-tuning process generally commences with the highest tier of the PyraNet dataset. Each tier within the PyraNet dataset is further categorized according to code complexity. Specifically, the process begins with data entries of `basic` complexity within the top tier, followed by `intermediate`, `advanced`, and `expert` complexity levels in sequence. This hierarchical structure is maintained across all tiers as the fine-tuning progresses downward through the dataset. By progressing through complexity levels in each tier, the model effectively leverages structured data to enhance its performance.

IV. EVALUATION AND DISCUSSION

A. Baseline LLMs

In this study, we aim to demonstrate the effectiveness of our proposed PyraNet dataset and the accompanying fine-tuning technique by conducting three distinct experiments and comparing the results with state-of-the-art models. We selected the CodeLlama-7b, CodeLlama-13b, and DeepSeek-Coder-7B models as our base models and fine-tuned them using the PyraNet dataset alongside our proposed fine-tuning methodology. For evaluation, we employed the VerilogEval platform to assess the performance of the models across all experiments.

B. Experiments

To thoroughly evaluate the effectiveness of our proposed PyraNet dataset and the accompanying fine-tuning technique, we designed a series of three experiments. These experiments aim to systematically assess the individual and combined impacts of the dataset and the fine-tuning approach on model performance. The experimental setup is as follows:

- **Baseline Experiment**: In the first experiment, we evaluated the CodeLlama and DeepSeek-Coder models by providing prompts without any fine-tuning. This established a baseline performance metric for comparison with subsequent experiments.

TABLE I
PYRANET VS SOTA MODELS ON VERILOGEVAL [10]

Model	Verilog-Machine			Verilog-Human		
	pass@1	pass@5	pass@10	pass@1	pass@5	pass@10
MG-Verilog-CodeLlama-7B [23]	54.5	60	63	-	-	-
RTL-Coder-DeepSeek [18]	61.2	76.5	81.8	41.6	50.1	53.4
OrigGen-DeepSeek [22]	74.1	82.4	85.7	54.4	60.1	64.2
codeLlama-7B-Instruct	41.9	46.1	46.8	19.2	23.0	25.0
codeLlama-7B-Instruct PyraNet-Dataset	58.0	62.9	67.8	44.2	50.0	55.7
codeLlama-7B-Instruct PyraNet-Architecture	62.9	69.2	74.1	48.0	53.2	57.6
codeLlama-13B-Instruct	48.6	54.5	57.3	32.6	35.8	39.1
codeLlama-13B-Instruct PyraNet-Dataset	58.7	63.6	67.8	45.5	51.2	57.0
codeLlama-13B-Instruct PyraNet-Architecture	69.2	73.4	78.3	50.6	55.1	58.3
DeepSeek-Coder-7B-Instruct-v1.5	55.2	72.7	76.9	32.6	43.5	49.3
DeepSeek-Coder-7B-Instruct-v1.5 PyraNet-Dataset	73.4	81.8	86.7	53.8	60.8	63.4
DeepSeek-Coder-7B-Instruct-v1.5 PyraNet-Architecture	77.6	84.6	89.5	58.3	62.8	67.9

TABLE II
PRE-TRAINED LLM ARCHITECTURES AND FINE-TUNING INFORMATION

Model	Layers	# of Heads	Head Size	Context Size	learning rate	# of epochs
CodeLlama-7B-Instruct	32	32	128	100,000	$2e^{-4}$	1, 2, 3
CodeLlama-13B-Instruct	40	40				
DeepSeek-Coder-7B-Instruct-v1.5	30	30		4000		

- **PyraNet-Only Fine-Tuning (PyraNet-Dataset)**: In the second experiment, the CodeLlama and DeepSeek-Coder models were fine-tuned using the PyraNet dataset without applying the specialized fine-tuning approach. This allowed us to isolate and assess the improvements attributable solely to the PyraNet dataset.
- **Combined PyraNet dataset and PyraNet Fine-Tuning Approach (PyraNet-Architecture)**: In the final experiment, we fine-tuned the CodeLlama and DeepSeek-Coder models using both the PyraNet dataset and our proposed fine-tuning technique. This combined approach was designed to evaluate the synergistic effects of the dataset and the fine-tuning methodology on model performance.

Through these experiments, we systematically evaluate the contributions of the PyraNet dataset and the fine-tuning technique, providing a comprehensive analysis of their impact compared to existing state-of-the-art models. The results are organized in Table I.

C. Fine-tuning Explained

As outlined in the preceding sections, we fine-tuned two models: one using the PyraNet dataset without applying the proposed fine-tuning technique, and the other utilizing both the PyraNet dataset and our proposed fine-tuning method.

In the first experiment, we fine-tuned the CodeLlama-7B, CodeLlama-13B, and DeepSeek-Coder-7B models using each available (data, description) pair from the dataset. Specifically, the descriptions were used as inputs to the models, and the corresponding Verilog code served as the output. Throughout this experiment, the learning rate was maintained at $2e^{-4}$, and the loss weights were set to 1.0. The fine-tuning method utilizes the LoRa technique [29], adhering to its standard training configurations. The baseline LLM architectures chosen for this study, along with the fine-tuning parameters utilized, are succinctly summarized in Table II.

For the second experiment, we employed a hierarchical fine-tuning approach. We began by fine-tuning the aforementioned models with data entries from the top layer of the PyraNet dataset. This phase consisted of four sequential fine-tuning steps, starting with data entries labeled `Basic`, followed by those labeled `Intermediate`, `Advanced`, and `Expert`.

In each step, descriptions were used as inputs and the corresponding Verilog codes as outputs, effectively implementing a curriculum learning strategy. During this initial fine-tuning phase, the loss weight was set to 1.0, reflecting the use of the highest quality data available.

In subsequent rounds of fine-tuning, we progressed to lower layers of the PyraNet dataset, continuing the curriculum learning approach for each layer. The primary modification in these rounds was the adjustment of the loss weights, as illustrated in Fig.1-b. By conducting multiple fine-tuning iterations with varying loss weights, we implemented the loss weighting approach. The fine-tuning process concluded upon completing the lowest layer of the PyraNet dataset. It is important to note that the learning rate was maintained at a constant value of $2e^{-4}$ for each round of fine-tuning throughout this experiment as well. This structured methodology allowed us to systematically assess the impact of both the hierarchical fine-tuning and loss weighting techniques on model performance, ensuring that the models effectively leveraged high-quality data while appropriately incorporating varying levels of complexity from the dataset.

D. Comparison and Explanation of the Results

To ensure a fair evaluation, the proposed PyraNet fine-tuning approach was applied to both the CodeLlama model, for comparison with MG-Verilog [23], and the DeepSeek-Coder model, for comparisons with RTL-Coder [18] and OriGen [22]. The results in Table I indicate that the proposed model achieved up to a 32.6% improvement in the $pass@k$ metric over the CodeLlama baseline models and up to 25.7% over the DeepSeek-Coder baseline model. When compared to state-of-the-art models, the proposed approach demonstrated an 11.1% improvement in the $pass@k$ metric on the Verilog-Machine dataset compared to MG-Verilog. For the Verilog-Human, the proposed model achieved improvements of up to 16.7% against RTL-Coder and 3.9% against OriGen, both of which are based on fine-tuning with DeepSeek-Coder.

It is worth noting that OriGen incorporates a Self-Reflection loop, which includes an additional round of error correction—a feature not considered in this work due to time constraints. This accounts for the relatively smaller improvements over OriGen. Nevertheless, integrating OriGen’s Self-Reflection loop with inference using the models fine-tuned with the proposed PyraNet architecture is expected to yield even greater performance gains, which will be performed in the future. All the gains achieved by the proposed PyraNet dataset and architecture are illustrated in Table III.

E. Dataset Quality Verification

As emphasized in [10], the integrity of the compiled dataset is crucial for the effectiveness of the fine-tuning process. Due to challenges in verifying each description and additional information incorporated into the PyraNet dataset via the GPT-4o-mini model, we adopted the verification methodology employed by the researchers in [10] to validate the labels generated by the GPT-4o-mini model. To assess the quality of the dataset, we randomly shuffled the codes, descriptions, and

TABLE III
PYRANET GAINS VS BASELINE MODEL AND SOTA

Model	VS	Verilog-Machine			Verilog-Human		
		$pass@1$	$pass@5$	$pass@10$	$pass@1$	$pass@5$	$pass@10$
codeLlama-7B-Instruct	vs Baseline	16.1	16.8	21	25	27	30.7
PyraNet-Dataset	vs MG-Verilog	3.5	2.9	4.8	-	-	-
codeLlama-7B-Instruct	vs Baseline	21	23.1	27.3	28.8	30.2	32.6
PyraNet-Architecture	vs MG-Verilog	8.4	9.2	11.1	-	-	-
codeLlama-13B-Instruct	vs Baseline	10.1	9.1	10.5	12.9	15.4	17.9
PyraNet-Dataset	vs MG-Verilog	4.2	3.6	4.8	-	-	-
codeLlama-13B-Instruct	vs Baseline	20.6	18.9	21	18	19.3	19.2
PyraNet-Architecture	vs MG-Verilog	14.7	13.4	15.3	-	-	-
DeepSeek-Coder-7B-Instruct-v1.5	vs Baseline	18.2	9.1	9.8	21.2	17.3	14.1
PyraNet-Dataset	vs RTL-Coder	12.2	5.3	4.9	12.2	10.7	10
	vs OriGen	-0.7	-0.6	1	-0.6	0.7	-0.8
DeepSeek-Coder-7B-Instruct-v1.5	vs Baseline	22.4	11.9	12.6	25.7	19.3	18.6
PyraNet-Architecture	vs RTL-Coder	16.4	8.1	7.7	16.7	12.7	14.5
	vs OriGen	3.5	2.2	3.8	3.9	2.7	3.7

TABLE IV
RESULTS FOR ERRONEOUS DATASET

Model	Verilog-Machine			Verilog-Human		
	$pass@1$	$pass@5$	$pass@10$	$pass@1$	$pass@5$	$pass@10$
CodeLlama-7B with erroneous dataset	43.3	42.2	50.3	21.1	25.6	28.8
CodeLlama-7B with correct dataset	58.0	62.9	67.8	44.2	50.0	55.7

ranking information among the data entries, thereby creating mismatched sets of codes, descriptions, and rankings within each row of the PyraNet dataset. We then proceeded to fine-tune the model using this intentionally distorted dataset. Table IV presents the results of fine-tuning the CodeLlama-7B model with both the flawed and accurate datasets. The findings clearly demonstrate that the model’s code generation capabilities are significantly compromised when trained on the erroneous dataset. This outcome verifies the accuracy of the labels and underscores the critical importance of dataset quality in the fine-tuning process. Furthermore, the results demonstrate that the accuracy of the fine-tuned model decreases when trained on the erroneous dataset compared to training solely on the PyraNet dataset without the proposed fine-tuning technique. Consequently, we decided to forgo the fine-tuning process using the proposed method, as it is evident that an erroneous dataset would similarly impair the effectiveness of the fine-tuning technique and result in reduced model performance.

V. CONCLUSION AND FUTURE WORKS

In this study, we present a novel structured dataset, PyraNet, alongside its associated fine-tuning architecture. This architecture addresses the limitations of previous studies by achieving improved performance across diverse Verilog code types. The PyraNet fine-tuning framework employs advanced techniques such as loss weighting and curriculum learning to mitigate the challenges posed by the available datasets. Evaluation results reveal that the PyraNet dataset and its proposed architecture achieve the anticipated benefits, demonstrating up to a 32.6% improvement in the $pass@k$ metric. We suggest that there is substantial scope for further advancements, both through the development of more comprehensive and detailed datasets and by exploring alternative fine-tuning architectures.

ACKNOWLEDGMENT

This work is supported by the National Science Foundation under Grant No. 2434247. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the funding agencies.

REFERENCES

- [1] Vaswani et al. Attention is all you need. 2017.
- [2] Radford et al. Improving language understanding by generative pre-training. 2018.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [4] Cohen et al. Lamda: Language models for dialog applications. In *arXiv*. 2022.
- [5] Md Rubel Ahmed, Bardia Nadimi, and Hao Zheng. System-on-chip message flow mining with masked-language models. In *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*, pages 496–500, 2023.
- [6] Deepak Vungarala, Sakila Alam, Arnob Ghosh, and Shaahin Angizi. Spicepilot: Navigating spice code generation and simulation with ai guidance, 2024.
- [7] Seyedsina Nabavirazavi, Rahim Taheri, Mohammad Shojafar, and Sundararaja Sitharama Iyengar. Impact of aggregation function randomization against model poisoning in federated learning. *TrustCom Proceedings Series*, pages 165–172, United States, May 2024. Institute of Electrical and Electronics Engineers Inc.
- [8] Dessouky et al. HardFails: Insights into Software-Exploitable hardware bugs. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 213–230, Santa Clara, CA, August 2019. USENIX Association.
- [9] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. Benchmarking large language models for automated verilog rtl code generation. pages 1–6. EDAA.
- [10] M. J. Liu, N. Pinckney, B. Khailany, and H. X. Ren. Invited paper: VerilogEval: Evaluating large language models for verilog code generation. *2023 IEEE/ACM International Conference on Computer Aided Design, ICCAD*, 2023.
- [11] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. Chipgpt: How far are we from natural language hardware design. *arXiv preprint arXiv:2305.14019*, 2023.
- [12] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RtlM: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 722–727. IEEE.
- [13] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. Chip-chat: Challenges and opportunities in conversational hardware design. In Andrew B. Kahng and Siddharth Garg, editors, *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, pages 1–6, [Piscataway, NJ], 2023. IEEE. Literaturangaben.
- [14] Fu et al. Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models, 2023.
- [15] Liu et al. Chipnemo: Domain-adapted llms for chip design, 2023.
- [16] Yun-Da Tsai, Mingjie Liu, and Haoxing Ren. Rtlfixer: Automatically fixing rtl syntax errors with large language models, 2024.
- [17] Thakur et al. Autochip: Automating hdl generation using llm feedback, 2023.
- [18] Liu et al. Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution, 2024.
- [19] Chang et al. Improving large language model hardware generating quality through post-LLM search. In *Machine Learning for Systems 2023*, 2023.
- [20] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.
- [21] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning, ICML '09*, pages 41–48, New York, NY, USA, 2009. Association for Computing Machinery.
- [22] Fan Cui et al. Orogen:enhancing rtl code generation with code-to-code augmentation and self-reflection, 2024.
- [23] Yongan Zhang, Zhongzhi Yu, Yonggan Fu, Cheng Wan, and Yingyan Celine Lin. Mg-verilog: Multi-grained dataset towards enhanced llm-assisted verilog generation, 2024.
- [24] Bardia Nadimi and Hao Zheng. A multi-expert large language model architecture for verilog code generation. pages 1–5, June 2024.
- [25] Yang Zhao et al. Codev: Empowering llms for verilog generation through multi-level summarization. *ArXiv*, abs/2407.10424, 2024.
- [26] Zehua Pei, Huiling Zhen, Mingxuan Yuan, Yu Huang, and Bei Yu. BetterV: Controlled verilog generation with discriminative guidance. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 40145–40153. PMLR, 21–27 Jul 2024.
- [27] Mingzhe Gao et al. Autovcoder: A systematic framework for automated verilog code generation using llms, 2024.
- [28] Stephen Williams. Icarus verilog.
- [29] Edward J. Hu et al. Lora: Low-rank adaptation of large language models, 2021.